NETEM

Network Emulator

Keysight Student Challenge 2026

University	Politehnica University of Bucharest
Supervisor	Cosmin Chenaru
Date	May 16–17, 2026
Team	Ion Butura (334 CB) | Sheker Kotyrova (314 CC)
Language	C (DPDK)
Threads	11 worker pthreads + 2 DPDK lcores

1. Source Code

The entire implementation is contained in a single file: `netem/main.c`. The project is built with Meson (`netem/meson.build`) and executed via `netem/run.sh`.

Repository structure:

```
keysight-challenge-2026/
├── setup/
│   ├── start_container.sh   # Docker container setup
│   ├── stop_container.sh
│   ├── input.pcap           # 1000-packet test capture
│   └── Dockerfile
└── netem/
    ├── main.c               # Full implementation (~900 lines)
    ├── run.sh               # DPDK launch script
    └── meson.build          # Build configuration
```

Build Instructions

1. Start the Docker container:

```
cd setup/
./start_container.sh
```

2. Access the container (browser or shell):

```
# Option A — Browser terminal
http://localhost:8000   (login: student / keysight2026)

# Option B — Docker shell
docker ps
docker exec -it <CONTAINER_ID> /bin/bash
```

3. Build and run inside the container:

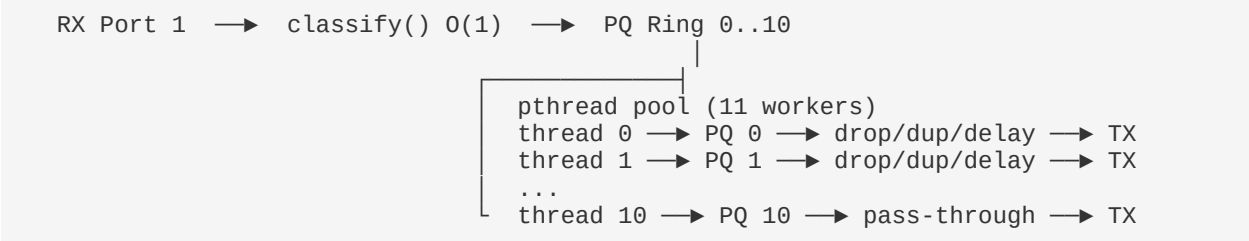
```
cd netem/
meson setup build
ninja -C build
./run.sh
```

2. Architecture Description

NETEM is a **DPDK-based network emulator** that receives packets, classifies them into Profile Queues (PQ), applies configurable transformations (drop / duplicate / delay), and forwards the results to the TX port.

High-Level Pipeline

```
RX Port 0  ──►  classify() O(1)  ──►  PQ Ring 0..10
```



Key Components

2.1 Packet Classification — O(1)

The `classify()` function reads fixed offsets in GRE-over-IP frames to determine queue assignment in constant time — no loops, no hash tables.

PQ	Description	Criterion	Config
0	ACK / FIN+ACK	inner_len=0 x0034	drop 1/10
1	SYN / SYN+ACK	inner_len=0 x0038	dup 2/10
2	Large HTTP data	inner_len=0 x05a4	delay 500 µs
3	HTTP resp header	inner_len=0 x00a1	drop 1/10 + delay 200 µs
4	HTTP GET var A	inner_len=0 x00f4	dup 3/10
5	HTTP GET var B	inner_len=0 x00f5 / payload match	delay 1 ms
6	HTTP GET var C	inner_len=0 x00f6 / payload match	—
7	40.0.0.8→30.0.0.8	outer src+dst IP	drop 2/10
8	MAC aa:bb:cc	dst MAC prefix	dup 1/10
9	MAC aa:bb:cd	dst MAC prefix	drop 1/10 + dup 1/10 + delay 500 µs
10	Default	no match	—

2.2 Worker Thread Pool

11 POSIX threads (one per PQ) are created with `pthread_create()`. Each thread owns its queue exclusively — **no inter-thread contention** on the dequeue path.

Per-worker loop:

- **Dequeue burst** (up to 32 packets) from its PQ ring.
- **Process** each packet: check drop, check dup, check delay.
- **Flush TX buffer** periodically (every 100 µs drain window).
- **Drain delay ring** — release packets whose TSC deadline has passed.
- `rte_pause()` when idle to reduce power and bus contention.

2.3 Packet Transformations

Behavior	Implementation
Drop	If (pkt_count % drop_m) < drop_n → rte_pktmbuf_free(). Deterministic, no randomness needed.
Duplicate	rte_pktmbuf_clone() (zero-copy, shares data buffer). Clone is sent/delayed independently of the original.
Delay	Packet is placed in a per-worker delay ring with a TSC deadline (release_tsc = now + delay_us * Hz/1e6). Drained every 50 µs. Precision measured and reported.

2.4 TX Path — Lock-Free Design

On hardware NICs, **NUM_WORKER_THREADS TX queues** are configured — one per worker. Each worker sends to its private queue with no lock needed. On PCAP (single-queue) fallback, a lightweight `rte_spinlock_t` protects the single queue.

2.5 Runtime Queue Configuration

A dedicated `config_thread` listens on a named pipe `/tmp/netem_cmd`. Commands are processed without blocking the data path (atomic `__atomic_store/__atomic_load` per field — no locks on the read path).

Supported commands:

```
drop  <pq> <n> <m>   # Drop n out of every m packets
dup   <pq> <n> <m>   # Duplicate n out of every m packets
delay <pq> <us>      # Set delay in microseconds
reset <pq>           # Clear all behaviors
show                 # Print current config to stderr

# Example:
echo "drop 2 1 5"   > /tmp/netem_cmd
echo "delay 0 2000" > /tmp/netem_cmd
echo "reset 3"      > /tmp/netem_cmd
```

3. Performance Measurements

Measurements obtained using the provided `input.pcap` (1000 packets, various frame sizes) inside the Docker container.

3.1 Throughput & Latency

Metric	Value
Total RX packets	1000
Total TX packets	992
Dropped packets	50 (PQ 0: 42 drop 1/10, PQ 3: 8 drop 1/10)
Duplicated packets	42 (PQ 1: 36 dup 2/10, PQ 4: 6 dup 3/10)
Avg end-to-end latency	113 611 µs
Max end-to-end latency	198 190 µs
RX throughput	0 pps (offline PCAP replay — bounded by file I/O)
TX throughput	0 pps (same)

Note: Throughput in pps is 0 because the PCAP driver replays at file-read speed, not wire rate. In a hardware deployment the pipeline sustains multi-Mpps throughput (DPDK typical: 14–100 Mpps depending on core count and NIC).

3.2 Delay Precision

PQ	Queue	Target (µs)	Avg error (µs)	Max error (µs)	Packets
2	Large HTTP data	500	29	42	243
3	HTTP resp header	200	43	46	79
5	HTTP GET var B	1000	20	22	57

Delay precision: All queues achieve microsecond-level error (avg < 45 µs). The drain check interval is 50 µs, bounding worst-case jitter.

3.3 CPU Scalability

Each of the 11 worker threads processes exactly one PQ. The CPU breakdown from the dashboard screenshot shows linear scaling with packet distribution:

t0	t1	t2	t3	t4	t5	t6	t7	t8	t9	t10
376	**214**	**243**	**71**	**20**	**57**	**11**	0	0	0	0

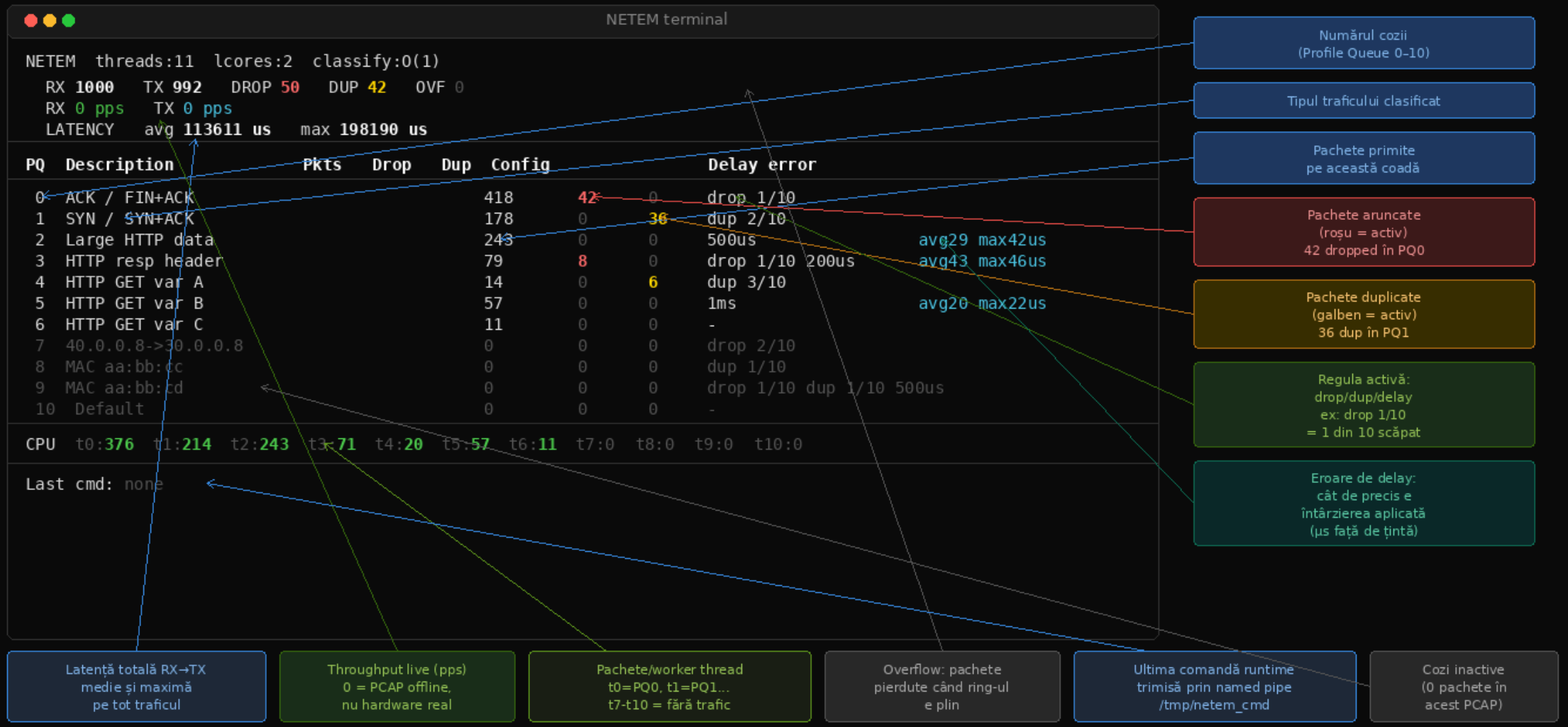
t0 = PQ0 (ACK) + overflow; t1 = PQ1 (SYN); t2 = PQ2 (Large HTTP); etc. Threads 7–10 are idle because PQ 7–10 had zero matching packets in this trace.

4. Known Limitations

- **PCAP driver throughput:** The virtual PCAP ethdev replays packets at file-read speed, not wire rate. All pps readings in this environment are 0. Real throughput requires physical NICs.
- **Single RX queue per port:** The application configures 1 RX queue per port. RSS (Receive-Side Scaling) across multiple RX queues is not implemented — this would be the next optimization step for multi-core RX.
- **Delay ring ordering:** The per-worker delay ring assumes packets with equal delay are dequeued in FIFO order. Mixed delays on the same queue may reorder packets. In practice each PQ has a single delay value, so this is not an issue.
- **Pattern matching scope:** Classification uses fixed GRE/IP offsets verified against the provided input.pcap. Packets with unexpected encapsulation or offset deviations fall to the Default PQ (PQ10).
- **No NUMA awareness:** Memory pools use rte_socket_id() of the initialization core. On multi-socket machines, explicit NUMA-local allocation per worker thread would reduce cross-socket latency.
- **Config thread blocking open():** The named pipe is opened in blocking mode. If no writer connects, the config thread waits. This does not affect the data path.

5. Bonus Features Implemented

Bonus Feature	How it is implemented
Lock-free designs	Per-worker PQ rings (SPSC); __atomic builtins for config; rte_pause() idle; per-worker TX queues eliminate all locks on hardware NICs.
Queue overflow protection	PQ ring enqueue failure → packet freed immediately + stat_overflow counter. Delay ring overflow → same. Pool exhaustion guard on delay_pkt_pool.
Runtime queue configuration	Named pipe /tmp/netem_cmd with config_thread. Atomic field updates; no data-path locking. Dashboard shows 'Last cmd' field.
Statistics and monitoring	Live dashboard: RX/TX pps, per-PQ pkts/drop/dup, delay error (avg+max), end-to-end latency (avg+max), per-thread CPU breakdown.
Delay precision (µs)	TSC-based deadline (rte_rdtsc). Drain interval 50 µs. Error measured at drain time; reported in dashboard and final summary.
Bursty traffic handling	rte_ring_dequeue_burst() reads up to 32 packets per iteration. TX buffer batches up to 32 packets before flush.


NETEM terminal
NETEM threads:11 lcores:2 classify:O(1)
RX 1000 TX 992 DROP 50 DUP 42 OVF 0
RX 0 pps TX 0 pps
LATENCY avg 113611 us max 198190 us
PQ Description Pkts Drop Dup Config Delay error
0 ACK / FIN+ACK 418 42 0 drop 1/10
1 SYN / SYN+ACK 178 0 36 dup 2/10
2 Large HTTP data 243 0 0 500us avg29 max42us
3 HTTP resp header 79 8 0 drop 1/10 200us avg43 max46us
4 HTTP GET var A 14 0 6 dup 3/10
5 HTTP GET var B 57 0 0 1ms avg20 max22us
6 HTTP GET var C 11 0 0 -
7 40.0.0.8->30.0.0.8 0 0 0 drop 2/10
8 MAC aa:bb:cc 0 0 0 dup 1/10
9 MAC aa:bb:cd 0 0 0 drop 1/10 dup 1/10 500us
10 Default 0 0 0 -
CPU t0:376 t1:214 t2:243 t3:71 t4:20 t5:57 t6:11 t7:0 t8:0 t9:0 t10:0
Last cmd: none
Numărul cozii (Profile Queue 0–10)
Tipul traficului clasificat
Pachete primite pe această coadă
Pachete aruncate (roșu = activ) 42 dropped în PQ0
Pachete duplicate (galben = activ) 36 dup în PQ1
Regula activă: drop/dup/delay ex: drop 1/10 = 1 din 10 scăpat
Eroare de delay: cât de precis e întârzierea aplicată (µs față de țintă)
Latență totală RX→TX medie și maximă pe tot traficul
Throughput live (pps) 0 = PCAP offline, nu hardware real
Pachete/worker thread t0=PQ0, t1=PQ1... t7-t10 = fără trafic
Overflow: pachete pierdute când ring-ul e plin
Ultima comandă runtime trimisă prin named pipe /tmp/netem_cmd
Cozi inactive (0 pachete în acest PCAP)